

maix.camera

maix.camera module, access camera device and get image from it

You can use `maix.camera` to access this module with MaixPy

This module is generated from `MaixPy` and `MaixCDK`

1. Module

No module

2. Enum

3. Variable

4. Function

4.1. list_devices

```
1 | def list_devices() -> list[str]
```

List all supported camera devices.

item	description
return	Returns the path to the camera device.
C++ definition code:	
<pre>1 std::vector<std::string> list_devices()</pre>	

4.2. set_regs_enable

```
1 | def set_regs_enable(enable: bool =
   | True) -> None
```

Enable set camera registers, default is false, if set to true, will not set camera registers, you can manually set registers by write_reg API.

item	description
param	enable: enable/disable set camera registers
C++ definition code:	
<pre>1 void set_regs_enable(bool enable = true)</pre>	

4.3. get_device_name

```
1 | def get_device_name() -> str
```

Get device name. Most of the time, the returned name is the name of the sensor.

C++ definition code:

```
1 | std::string get_device_name()
```

5. Class

5.1. Camera

Camera class

C++ definition code:

```
1 | class Camera
```

5.1.1. __init__

```
1 | def __init__(self, width: int = -1,
    height: int = -1, format:
    maix.image.Format = ..., device: str
    = None, fps: float = -1, buff_num:
    int = 3, open: bool = True, raw:
    bool = False) -> None
```

Construct a new Camera object.\nMaximum resolution support 2560x1440.

item	description
type	func

item	description
param	width: camera width, default is -1, means auto, mostly means max width of camera support height: camera height, default is -1, means auto, mostly means max height of camera support format: camera output format, default is image.Format.FMT_RGB888 device: camera device path, you can get devices by list_devices method, by default(value is NULL(None in MaixPy)) means the first device fps: camera fps, default is -1, means auto, mostly means max fps of camera support buff_num: camera buffer number, default is 3, means 3 buffer, one used by user, one used for cache the next frame, more than one buffer will accelerate image read speed, but will cost more memory. open: If true, camera will automatically call open() after creation. default is true. raw: If true, you can use read_raw() to capture the raw image output from the sensor.
static	False



C++ definition code:

```
1 | Camera(int width = -1, int height  
   | = -1, image::Format format =  
   | image::FMT_RGB888, const char  
   | *device = nullptr, double fps =  
   | -1, int buff_num = 3, bool open =  
   | true, bool raw = false)
```

5.1.2. get_ch_nums

```
1 | def get_ch_nums(self) -> int
```

Get the number of channels supported by the camera.

item	description
type	func
return	Returns the maximum number of channels.
static	False

C++ definition code:

```
1 | int get_ch_nums()
```

5.1.3. open

```
1 | def open(self, width: int = -1,  
   | height: int = -1, format:  
   | maix.image.Format = ..., fps: float  
   | = -1, buff_num: int = -1) ->
```

Open camera and run

item	description
type	func
param	width: camera width, default is -1, means auto, mostly means max width of camera support height: camera height, default is -1, means auto, mostly means max height of camera support format: camera output format, default same as the constructor's format argument fps: camera fps, default is -1, means auto, mostly means max fps of camera support buff_num: camera buffer number, default is 3, means 3 buffer, one used by user, one used for cache the next frame, more than one buffer will accelerate image read speed, but will cost more memory.
return	error code, err::ERR_NONE means success, others means failed
static	False

C++ defination code:

```

1 | err::Err open(int width = -1, int
    height = -1, image::Format format
    = image::FMT_INVALID, double fps
    = -1, int buff_num = -1)

```

5.1.4. read

```

1 | def read(self, buff: capsule = None,
    buff_size: int = 0, block: bool =
    True, block_ms: int = -1) ->
    maix.image.Image

```

Get one frame image from camera buffer, must call open method before read.\nIf open method not called, will call it automatically, if open failed, will throw exception!\nSo call open method before read is recommended.

item	description
type	func
param	buff: buffer to store image data, if buff is nullptr, will alloc memory automatically. In MaixPy, default to None, you can create a image.Image object, then pass img.data() to buff. block: block read, default is true, means block util read image successfully, if set to false, will return nullptr if no image in buffer block_ms: block read timeout

item	description
return	image::Image object, if failed, return nullptr, you should delete if manually in C++
static	False

C++ definition code:

```
1 | image::Image *read(void *buff =
    nullptr, size_t buff_size = 0,
    bool block = true, int block_ms =
    -1)
```

5.1.5. read_raw

```
1 | def read_raw(self) ->
    maix.image.Image
```

Read the raw image and obtain the width, height, and format of the raw image through the returned Image object.

item	description
type	func
note	The raw image is in a Bayer format, and its width and height are affected by the driver. Modifying the size and format is generally not allowed.
return	image::Image object, if failed, return nullptr, you should delete if manually in C++

item	description
static	False
C++ definition code:	
<pre>1 image::Image *read_raw()</pre>	

5.1.6. clear_buff

```
1 | def clear_buff(self) -> None
```

Clear buff to ensure the next read image is the latest image

item	description
type	func
static	False

C++ definition code:

```
1 | void clear_buff()
```

5.1.7. skip_frames

```
1 | def skip_frames(self, num: int) ->
    None
```

Read some frames and drop, this is usually used avoid read not stable image when camera just opened.

item	description
type	func
param	num : number of frames to read and drop
static	False

C++ definition code:

```
1 | void skip_frames(int num)
```

5.1.8. close

```
1 | def close(self) -> None
```

Close camera

item	description
type	func
static	False

C++ definition code:

```
1 | void close()
```

5.1.9. add_channel

```
1 | def add_channel(self, width: int =
    -1, height: int = -1, format:
    maix.image.Format = ..., fps: float
    = -1, buff_num: int = 3, open: bool
```

```
= True) -> Camera
```

Add a new channel and return a new Camera object, you can use close() to close this channel.

item	description
type	func
param	width: camera width, default is -1, means auto, mostly means max width of camera support height: camera height, default is -1, means auto, mostly means max height of camera support format: camera output format, default is RGB888 fps: camera fps, default is -1, means auto, mostly means max fps of camera support buff_num: camera buffer number, default is 3, means 3 buffer, one used by user, one used for cache the next frame, more than one buffer will accelerate image read speed, but will cost more memory. open: If true, camera will automatically call open() after creation. default is true.
return	new Camera object
static	False

C++ definition code:

```
1 | camera::Camera *add_channel(int  
   | width = -1, int height = -1,  
   | image::Format format =  
   | image::FMT_RGB888, double fps =  
   | -1, int buff_num = 3, bool open =  
   | true)
```

5.1.10. is_opened

```
1 | def is_opened(self) -> bool
```

Check if camera is opened

item	description
type	func
return	true if camera is opened, false if not
static	False

C++ definition code:

```
1 | bool is_opened()
```

5.1.11. is_closed

```
1 | def is_closed(self) -> bool
```

check camera device is closed or not

item	description
type	func
return	closed or not, bool type
static	False

C++ definition code:

```
1 | bool is_closed()
```

5.1.12. width

```
1 | def width(self) -> int
```

Get camera width

item	description
type	func
return	camera width
static	False

C++ definition code:

```
1 | int width()
```

5.1.13. height

```
1 | def height(self) -> int
```

Get camera height

item	description
type	func
return	camera height
static	False

C++ definition code:

```
1 | int height()
```

5.1.14. fps

```
1 | def fps(self) -> float
```

Get camera fps

item	description
type	func
return	camera fps
static	False

C++ definition code:

```
1 | double fps()
```

5.1.15. format

```
1 | def format(self) ->
    maix.image.Format
```

Get camera output format

item	description
type	func
return	camera output format, image::Format object
static	False

C++ definition code:

```
1 | image::Format format()
```

5.1.16. buff_num

```
1 | def buff_num(self) -> int
```

Get camera buffer number

item	description
type	func
return	camera buffer number
static	False

C++ definition code:

```
1 | int buff_num()
```

5.1.17. hmirror

```
1 | def hmirror(self, value: int = -1) -> int
```

Set/Get camera horizontal mirror

item	description
type	func
return	camera horizontal mirror
static	False

C++ defination code:

```
1 | int hmirror(int value = -1)
```

5.1.18. vflip

```
1 | def vflip(self, value: int = -1) -> int
```

Set/Get camera vertical flip

item	description
type	func
return	camera vertical flip
static	False

C++ defination code:

```
1 | int vflip(int value = -1)
```


5.1.19. device

```
1 | def device(self) -> str
```

Get camera device path

item	description
type	func
return	camera device path
static	False

C++ defination code:

```
1 | std::string device()
```

5.1.20. write_reg

```
1 | def write_reg(self, addr: int, data: int, bit_width: int = 8) -> maix.err.Err
```

Write camera register

item	description
type	func
param	addr : register address data : register data bit_width : register data bit width, default is 8

item	description
return	error code, err::ERR_NONE means success, others means failed
static	False

C++ definition code:

```
1 | err::Err write_reg(int addr, int
   | data, int bit_width = 8)
```

5.1.21. read_reg

```
1 | def read_reg(self, addr: int,
   | bit_width: int = 8) -> int
```

Read camera register

item	description
type	func
param	addr : register address bit_width : register data bit width, default is 8
return	register data, -1 means failed
static	False

C++ definition code:

```
1 | int read_reg(int addr, int
   | bit_width = 8)
```

5.1.22. show_colorbar

```
1 | def show_colorbar(self, enable:  
    bool) -> maix.err.Err
```

Camera output color bar image for test

item	description
type	func
param	enable : enable/disable color bar
return	error code, err::ERR_NONE means success, others means failed
static	False

C++ definition code:

```
1 | err::Err show_colorbar(bool  
    enable)
```

5.1.23. get_channel

```
1 | def get_channel(self) -> int
```

Get channel of camera

item	description
type	func
return	channel number
static	False

C++ definition code:

```
1 | int get_channel()
```

5.1.24. set_resolution

```
1 | def set_resolution(self, width: int,  
    height: int) -> maix.err.Err
```

Set camera resolution

item	description
type	func
param	width : new width height : new height
return	error code, err::ERR_NONE means success, others means failed
static	False

C++ definition code:

```
1 | err::Err set_resolution(int  
    width, int height)
```

5.1.25. set_fps

```
1 | def set_fps(self, fps: float) ->  
    maix.err.Err
```

Set camera fps

item	description
type	func
param	fps : new fps
return	error code, err::ERR_NONE means success, others means failed
static	False

C++ definition code:

```
1 | err::Err set_fps(double fps)
```

5.1.26. exposure

```
1 | def exposure(self, value: int = -1)
   | -> int
```

Set/Get camera exposure

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it.
param	value : exposure time. unit: us If value == -1, return exposure time. If value != 0, set and return exposure time.
return	camera exposure time

item	description
static	False
C++ definition code:	
<pre>1 int exposure(int value = -1)</pre>	

5.1.27. gain

```
1 | def gain(self, value: int = -1) ->
    int
```

Set/Get camera gain

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it.
param	value: camera gain. If value == -1, returns camera gain. If value != 0, set and return camera gain.
return	camera gain
static	False
C++ definition code:	

```
1 | int gain(int value = -1)
```

5.1.28. luma

```
1 | def luma(self, value: int = -1) ->  
    int
```

Set/Get camera luma

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it.
param	value : luma value, range is [0, 100] If value == -1, returns luma value. If value != 0, set and return luma value.
return	returns luma value
static	False

C++ defination code:

```
1 | int luma(int value = -1)
```

5.1.29. constrast

```
1 | def constrast(self, value: int = -1)  
    -> int
```

Set/Get camera constrast

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it.
param	value: constrast value, range is [0, 100] If value == -1, returns constrast value. If value != 0, set and return constrast value.
return	returns constrast value
static	False

C++ defination code:

```
1 | int contrast(int value = -1)
```

5.1.30. saturation

```
1 | def saturation(self, value: int =  
   | -1) -> int
```

Set/Get camera saturation

item	description
type	func

item	description
attention	This method will affect the isp and thus the image, so please be careful with it.
param	value: saturation value, range is [0, 100] If value == -1, returns saturation value. If value != 0, set and return saturation value.
return	returns saturation value
static	False

C++ definition code:

```
1 | int saturation(int value = -1)
```

5.1.31. awb_mode

```
1 | def awb_mode(self, value: int = -1)
   -> int
```

Set/Get white balance mode (deprecated interface)

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it. This interface may be deprecated

item	description
	in the future, and there may be incompatibilities in the definition of the parameters of the new interface
param	value: value = 0, means set white balance to auto mode, value = 1, means set white balance to manual mode, default is auto mode.
return	returns awb mode
static	False
C++ definition code:	
<pre>1 int awb_mode(int value = -1)</pre>	

5.1.32. set_awb

```
1 | def set_awb(self, mode: int = -1) ->
   | int
```

Set/Get white balance mode

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it.

item	description
param	value: value = 0, means set white balance to manual mode, value = 1, means set white balance to auto mode, default is auto mode.
return	returns awb mode
static	False

C++ definition code:

```
1 | int set_awb(int mode = -1)
```

5.1.33. exp_mode

```
1 | def exp_mode(self, value: int = -1)
   -> int
```

Set/Get exposure mode (deprecated interface)

item	description
type	func
attention	This method will affect the isp and thus the image, so please be careful with it. This interface may be deprecated in the future, and there may be incompatibilities in the definition of the parameters of the new interface

item	description
param	value: value = 0, means set exposure to auto mode, value = 1, means set exposure to manual mode, default is auto mode.
return	returns exposure mode
static	False

C++ definition code:

```
1 | int exp_mode(int value = -1)
```

5.1.34. set_windowing

```
1 | def set_windowing(self, roi:
   | list[int]) -> maix.err.Err
```

Set window size of camera

item	description
type	func
param	roi: Support two input formats, [x,y,w,h] set the coordinates and size of the window; [w,h] set the size of the window, when the window is centred.
return	error code
static	False

C++ definition code:

```
1 | err::Err  
   | set_windowing(std::vector<int>  
   | roi)
```

